

Azure Service Bus – Sending Messages with Application Properties

1. What is the Purpose of This Example?

In this example, we will:

1. Create a Service Bus message in a C# console application.
2. Add a **custom/application property** to the message.
3. Send the message to an Azure Service Bus topic.
4. The topic has a subscription called `S1`.
5. `S1` has a **Correlation Filter**:

```
City = Mumbai
```

1. Because our message contains:

```
City = Mumbai
```

the message should be delivered to subscription `S1`.

2. Overall Flow

```
C# Console Application
    ↓
Create Service Bus Message
    ↓
Add Application Property
City = Mumbai
    ↓
Send Message to Topic
    ↓
Topic
    ↓
Subscription S1
    ↓
Correlation Filter
City = Mumbai
    ↓
MATCH
```

↓
Message delivered to S1

3. Prerequisite – Correlation Filter

Before sending the message, subscription `S1` already has a correlation filter.

The filter is:

City = Mumbai

This means S1 will receive only messages where the application property matches:

City = Mumbai

For example:

Message 1:

City = Mumbai

→ S1 receives ✓

Message 2:

City = Bangalore

→ S1 doesn't receive ✗

4. Create the User Class

We first create a simple C# class:

```
public class User
{
    public string Username { get; set; }
    public string City { get; set; }
}
```

This class contains two properties:

```
Username  
City
```

5. Create a User Object

Now create an object:

```
var user1 = new User  
{  
    Username = "John",  
    City = "Mumbai"  
};
```

The object contains:

```
Username = John  
City      = Mumbai
```

6. Serialize the Object

We can serialize the object into JSON.

Example:

```
var json = JsonConvert.SerializeObject(user1);
```

The resulting JSON will look like:

```
{  
  "Username": "John",  
  "City": "Mumbai"  
}
```

This JSON can be used as the message body.

7. Create the Service Bus Message

Now create a Service Bus message:

```
var message = new ServiceBusMessage(json);
```

Here:

```
json  
↓  
ServiceBusMessage
```

The message body contains:

```
{  
  "Username": "John",  
  "City": "Mumbai"  
}
```

8. Add an Application Property

This is the most important part of this example.

We add a custom application property:

```
message.ApplicationProperties.Add("City", "Mumbai");
```

This adds:

```
Property Name  = City  
Property Value = Mumbai
```

The message now contains two different types of information:

Message Body

```
{  
  "Username": "John",  
  "City": "Mumbai"  
}
```

Application Property

```
City = Mumbai
```

9. Why Do We Need the Application Property?

This is very important.

The correlation filter does **not simply inspect the JSON message body**.

Our subscription filter is:

```
City = Mumbai
```

The `City` value used by the correlation filter is the **application property**.

Therefore, we explicitly add:

```
message.ApplicationProperties.Add("City", "Mumbai");
```

Think of it like this:

```
Message  
|  
├── Body  
│   ├── Username = John  
│   └── City = Mumbai  
└── Application Properties  
    └── City = Mumbai ← Filter checks this
```

10. Send the Message

Once the message is created and the application property is added, send it to the topic:

```
await serviceBusSender.SendMessageAsync(message);
```

The flow is:

```
ServiceBusSender
  ↓
SendMessageAsync()
  ↓
Azure Service Bus Topic
```

11. How Azure Service Bus Processes the Message

The message arrives at the topic.

Suppose the topic has subscription `S1`.

S1 has:

```
Correlation Filter:

City = Mumbai
```

The incoming message contains:

```
Application Property:

City = Mumbai
```

Azure Service Bus compares them:

```
Filter Value:
Mumbai
```

Message Property:
Mumbai

Result:

Mumbai == Mumbai
↓
TRUE

Therefore:

Message → S1 

12. Complete Example Code

A simplified version of the complete code is:

```
using Azure.Messaging.ServiceBus;
using Newtonsoft.Json;

public class User
{
    public string Username { get; set; }
    public string City { get; set; }
}

class Program
{
    static async Task Main(string[] args)
    {
        string connectionString = "...";
        string topicName = "firsttopic";

        var client = new ServiceBusClient(connectionString);

        var sender = client.CreateSender(topicName);

        var user1 = new User
        {
            Username = "John",
            City = "Mumbai"
        }
    }
}
```

```
};

var json = JsonConvert.SerializeObject(user1);

var message = new ServiceBusMessage(json);

message.ApplicationProperties.Add("City", "Mumbai");

await sender.SendMessageAsync(message);

Console.WriteLine("Message sent successfully.");
}
```

13. Important Difference: Message Body vs Application Property

This is a very important interview concept.

Message Body

The body contains the actual business data.

Example:

```
{
  "Username": "John",
  "City": "Mumbai"
}
```

Application Property

Application properties are additional metadata attached to the message.

Example:

```
City = Mumbai
```

They can be used by Service Bus filters.

So:


```
Message Body
  ↓
Business data

Application Properties
  ↓
Metadata / filtering information
```

14. Testing in Azure Portal

After running the C# application, go to:

```
Azure Portal
  ↓
Service Bus Namespace
  ↓
Topic
  ↓
Subscription S1
```

You should see:

```
Message Count = 1
```

This confirms that the message matched the correlation filter.

15. Verify the Message Using Service Bus Explorer

Open the message using **Service Bus Explorer**.

You should see the message body:

```
{
  "Username": "John",
  "City": "Mumbai"
}
```

You can also open **Message Properties**.

You should see:

```
City = Mumbai
```

This confirms that the application property was successfully attached to the message.

16. What If We Send City = Bangalore?

Suppose we change:

```
message.ApplicationProperties.Add("City", "Bangalore");
```

But S1 still has:

```
City = Mumbai
```

The comparison becomes:

```
Message Property:  
City = Bangalore  
  
Filter:  
City = Mumbai
```

They don't match.

Therefore:

```
Message → S1 ❌
```

17. Important Scenario

Suppose the message body contains:

```
{  
  "Username": "John",
```

```
"City": "Mumbai"
}
```

but we don't add the application property:

```
City = Mumbai
```

The correlation filter may not match simply because `City` exists inside the JSON body.

Why?

Because the filter is checking the **application property**, not automatically searching inside the JSON body.

Therefore, we need:

```
message.ApplicationProperties.Add("City", "Mumbai");
```

18. Real-Time Example

Consider an order processing system.

We have:

```
OrderTopic
```

Subscriptions:

```
MumbaiOrders
BangaloreOrders
DelhiOrders
```

Filters:

```
MumbaiOrders
→ City = Mumbai

BangaloreOrders
→ City = Bangalore
```

```
DelhiOrders  
→ City = Delhi
```

Application sends:

```
{  
  "OrderId": 101,  
  "Customer": "John",  
  "City": "Mumbai"  
}
```

And adds:

```
Application Property:  
  
City = Mumbai
```

Service Bus checks the subscriptions.

```
MumbaiOrders  
City = Mumbai  
↓  
MATCH ✓  
  
BangaloreOrders  
City = Bangalore  
↓  
NO MATCH ✗  
  
DelhiOrders  
City = Delhi  
↓  
NO MATCH ✗
```

Therefore, the message is delivered only to:

```
MumbaiOrders
```

19. Key Points to Remember

- `ServiceBusMessage` represents a message sent through Azure Service Bus.
- The **message body** contains the actual data.
- **Application properties** contain additional key-value metadata.
- Application properties can be used for message filtering.
- Example:

```
message.ApplicationProperties.Add("City", "Mumbai");
```

- A correlation filter can check:

```
City = Mumbai
```

- If the message property matches the filter, the message is delivered to that subscription.
- If it doesn't match, that subscription does not receive the message.
- The correlation filter does not automatically search inside the JSON message body.

20. Interview Answer

Question: How do you send a message with a custom property in Azure Service Bus?

Answer:

We create a `ServiceBusMessage`, add the custom property using `ApplicationProperties`, and then send the message using `SendMessageAsync()`.

Example:

```
var message = new ServiceBusMessage(json);

message.ApplicationProperties.Add(
    "City",
    "Mumbai"
);

await sender.SendMessageAsync(message);
```

If the subscription has a correlation filter:

```
City = Mumbai
```

the message will be delivered to that subscription.

Quick Revision

```
User Object
  ↓
Serialize to JSON
  ↓
Create ServiceBusMessage
  ↓
Add Application Property
City = Mumbai
  ↓
SendMessageAsync()
  ↓
Topic
  ↓
Subscription S1
  ↓
Correlation Filter
City = Mumbai
  ↓
MATCH
  ↓
Message delivered to S1
```

Remember This

Message body = business data

Application property = metadata that can be used for filtering

Correlation filter = checks equality of message properties